

Introduction

In this activity, we will apply machine learning to a simple question in population genetics. Specifically, we will simulate data under two simple demographic models using `msprime`. Then, we will calculate Site Frequency Spectra (SFS) from these data, and store them as numpy matrices. We will use these matrices to train a convolutional neural network (CNN) using tensorflow and keras. Finally, we will generate an independent test dataset and test the classifier. This is a simple example that could be solved without using machine learning, but it should illustrate the basic principles of using machine learning in python to analyze genomic data.

Simulating genetic data in `msprime`

There are many simulators we could use to simulate data, and different problems require different simulators. For example, when we want to simulate selection, SLiM (Haller and Messer, 2022) may be a good choice. In this simple example, we will use `msprime` (Baumdicker et al., 2022). `msprime` integrates well into python pipelines, since it is a python package. It is also used in `stdpopsim`, a library for standardized population genetic simulations for many species (Adrion et al., 2020). We will keep it simple for the point of illustration and simulate data under two models: a divergence-only model and a divergence-with-gene-flow model. Our question is: do these two sister species have a history of gene flow?

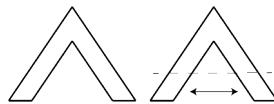


Figure 1: Demographic models

To simulate data:

```
python Machine_Learning_for_Population_Genetics_V2.py --simulate
```

Training the model

Now, we can train a machine learning algorithm to distinguish amongst the two models using the SFS as our data. We will use a Convolutional Neural Network (CNN).

To train a network with 2 hidden layers, a dropout rate of 0.2, 10 epochs, and a learning rate of 1e-05:

```
python Machine_Learning_for_Population_Genetics_V2.py --train \  
--layers 2 --dropout 0.2 --epochs 10 --learning_rate 1e-05
```

Experiment with the different parameters!

- What happens if you use only one layer? three layers?

- What happens if you change dropout to 0? 0.5?
- What happens if you increase the learning rate to 1e-03? decrease it to 1e-07?
- What happens if you decrease the number of epochs to 5? increase it to 15?

Testing the model on an independent dataset.

After you've settled on the number of layers, dropout, number of epochs, and learning rate you want to use, you need to test your network on an independent test set! Don't run this part until you've settled on your hyperparameters!

```
python Machine_Learning_for_Population_Genetics_V2.py --test
```